

A reliability-first stack for Dreambase's agent loops

My perspective: I prepared this proposal with AI-assisted research using public sources checked September 17, 2026. I approach it from business operations and practical tool-building. I have no access to Dreambase's internal systems or production agent-engineering experience.

Start from the architecture Dreambase already has

Dreambase documents a durable orchestrator coordinating specialist agents through bounded tool loops, with validation, recovery paths, and commit conditions. A Data Engineer produces reusable datasets; component-level DuckDB SQL runs against materialized Parquet; deterministic checks precede rendered review; and analytical lineage follows the result. Its API distinguishes MCP exploration from REST operations on known resources. [Engineering description](#), [product description](#), [API reference](#).

Those are documented properties, not proposals. My recommendation is to preserve that durable orchestrator and strengthen its contracts. A framework replacement would add migration risk without solving metric meaning, dataset grain, context isolation, schema change, or replayability.

Proposed stack and boundaries

I would put TypeScript at the orchestration and tool boundary. Each graph node should accept and return a versioned, runtime-validated object, using Zod or an equivalent JSON Schema implementation. A result such as `ok | failed | not_queryable | unavailable` makes recovery an explicit state transition. Persisted state should include tenant and submission IDs, context hashes, dataset version, budgets, tool receipts, model and prompt versions, and commit evidence. Static types aid refactoring; runtime validation protects the boundary where model output enters ordinary code.

Python is useful for offline dataset construction, statistical checks, and evaluation analysis. I would not split the online loop across languages without measured need. SQL remains the analytical language, and DuckDB the execution layer over materialized snapshots, matching Dreambase's documented design. Validation and serving can query the same bytes while source variability stays on the refresh side.

Decision	Proposed choice	Alternative	Tradeoff
Orchestration	Preserve the durable review graph	Replace it with a generic agent runtime	Preservation keeps replay and recovery semantics; replacement only makes sense after measured limitations
Online contracts	TypeScript plus runtime schemas	Prompt-only JSON conventions	More schema/version work, but malformed outputs fail before persistence
Data execution	DuckDB over versioned materialized snapshots	Query every source live	Snapshots are reproducible and reusable; freshness must be explicit
Python	Offline data and evaluation tooling when useful	Python in every online node	A narrower role reduces deployment and type-boundary complexity
Model selection	Route by task-specific evaluations	One default model or public benchmark ranking	More evaluation maintenance, but routing follows Dreambase's actual failure modes

Treat semantics as a first-class contract

A syntactically valid query can still be wrong. Every governed metric should carry at least: `metric_id`, semantic version, human definition, numerator and denominator, declared grain, allowed dimensions, timezone, currency or unit, null policy, source lineage, dataset version, `as_of` time, and owner. A change in grain or business definition is a breaking change even if the SQL still runs.

Consider a tiny sales fixture:

orders	amount
O-1	\$100
O-2	\$50

`order_lines` has two rows for O-1 and one for O-2. A direct join followed by `SUM(orders.amount)` returns **\$250** because O-1 is duplicated: $\$100 + \$100 + \$50$. The expected revenue at order grain is **\$150**, across **2 orders**. The safe plan either aggregates line data to one row per order before joining or calculates revenue from an order-grain relation. A validator can detect the risk by comparing declared grain with join cardinality, then run fixture invariants: row uniqueness at the declared key, \$150 total, and 2 distinct orders.

Validation should climb from cheap certainty to expensive judgment:

1. Parse the tool result and enforce the typed schema.
2. Check tenant scope, read-only SQL policy, referenced relations, output aliases, and component shape.
3. Execute against the exact dataset snapshot with row, memory, and time limits.
4. Test grain, null, range, reconciliation, freshness, and lineage invariants.
5. Render the accepted result and only then ask an LLM critic about semantic fit or visual clarity.

An LLM critic is valuable for questions code cannot fully answer, but it should never overrule a deterministic failure. The system should promise inspectable, rejectable work, not infallibility.

Context, tenancy, and change

The [role description](#) names cascading workspace, project, and query context. I would represent each layer as a typed, immutable input with provenance and precedence rules. A query-level definition can narrow a time range; it should not silently replace a workspace-level revenue definition. The assembled context gets a content hash stored with the run so an auditor can reconstruct what the agent saw.

Every tool call and persisted artifact should carry `workspace_id` and `project_id`; authorization must be enforced server-side rather than inferred from prompt context. Tests should attempt cross-workspace dataset handles, stale credentials, and mixed-context replays. Freshness is similarly explicit: expose snapshot creation time, source watermark, refresh status, and timezone to both the planner and user. "Latest" without those fields is not a reliable analytical instruction.

Schema drift should trigger classification, not improvisation. Additive nullable columns may pass after profile refresh. Removed or renamed columns, type changes, and key/cardinality changes should block affected components and identify dependents through lineage. Keep the last known-good artifact available with a visible stale state while the graph repairs or requests help.

Failure handling and release evidence

Give every node a deadline, attempt budget, and idempotency key derived from tenant, submission, operation, and payload hash. Retry transient failures with bounded backoff and jitter; do not retry invalid SQL unchanged. Refreshes can have external effects, so ambiguous outcomes require reconciliation before another write. An internal key cannot make a non-idempotent upstream API safe: without provider support or a verifiable receipt, stop for review. Exhausted budgets should end in `unavailable`. [Refresh semantics](#).

Model routing should be earned per task. Pin model, prompt, tools, and context-builder versions; evaluate planner, SQL author, semantic reviewer, and visual critic separately; and promote a routing change only when it clears the relevant suite. Cost and latency are constraints, while task accuracy and safe failure behavior are release gates. Shadow evaluation on saved, redacted traces can compare providers without silently changing production behavior.

The following matrix is illustrative; it contains proposed cases and gates, not measured results:

Task slice	Fixture	Deterministic gate	Review signal	Proposed release gate
Dataset planning	One-to-many sales join	Declared grain remains unique; totals reconcile	Plan covers stated intent	No critical invariant regression
SQL composition	Renamed revenue column	Binding and execution succeed or fail explicitly	Explanation identifies drift	100% safe handling of blocking drift cases
Context assembly	Conflicting timezone definitions	Precedence and provenance match contract	Answer states chosen timezone	Zero cross-tenant or silent-precedence failures
Recovery	Timeout after an ambiguous refresh	No duplicate side effect	Recovery message is actionable	Reconcile or stop; no blind mutation retries
Rendering	Long labels and empty series	Valid geometry and bindings	Critic flags unreadable output	No deterministic regression; review threshold set from labeled examples

Release suites should include golden fixtures, adversarial joins, empty data, schema drift, stale snapshots, timezone boundaries, prompt injection in source text, timeouts, and cross-tenant references. Human labels need written rubrics and disagreement review; otherwise judge scores can measure inconsistent preferences.

What I made executable

I built a Metric Reliability Lab alongside this paper. It runs real SQLite through `sql.js` in the browser, with fixed query plans, bound parameters, and synthetic sales fixtures. I chose SQLite to keep the work sample portable; I am not proposing it as a replacement for Dreambase's DuckDB layer. The validator compares candidate records and totals against a separate JavaScript implementation of the metric contract. Its receipt includes the contract version, snapshot identifier, fixed evaluation clock, SQL, parameters, rows, and each acceptance check.

The reproducible CLI evaluation contains nine scenario/plan combinations. The current results are:

Fixture and plan	Candidate / reference	Outcome
Line join / order-grain repair	\$250 / \$150; then \$150 / \$150	Blocked; then accepted
Equal-price orders, DISTINCT / order grain	\$100 / \$200; then \$200 / \$200	Blocked; then accepted
Missing tenant filter / scoped plan	\$1,050 / \$150; then \$150 / \$150	Blocked; then accepted
Correct query, 72-hour-old snapshot	\$150 / \$150	Blocked: 24-hour freshness limit
Unknown paid amount	\$100 / unknown	Blocked: NULL cannot imply zero

Fixture and plan	Candidate / reference	Outcome
Empty date interval	\$0 / \$0	Accepted: no eligible orders

Four accepted and five blocked outcomes match the fixed expectations. This is a measured fixture result, not a live-model benchmark. Both the SQL and reference calculation could share a misunderstood business definition; independent stakeholder review still matters. These cases do not prove arbitrary SQL correct, measure performance, or establish security. In particular, client-side workspace checks teach a scope invariant but cannot enforce authorization. The JSON receipt is inspectable, not signed or tamper-proof. A fixed evaluation clock makes the freshness cases reproducible without pretending the data is current.

My next learning experiment

In week one, I would build a small TypeScript loop over synthetic sales data with persisted run state and a versioned DuckDB snapshot. Week two adds 25-40 focused evaluation cases. Week three compares two model routes on the same cases, reporting quality, latency, cost, and failure classes. Week four injects timeouts and ambiguous refreshes and publishes the fixtures, rubric, and limitations. Replayed outputs can test control flow when provider access is unavailable, but cannot support claims about live model quality, latency, or cost.

My workbench also includes an auditor replay that rejects unsupported verdicts and invalid checklist identifiers. The SQL lab and response validator give me concrete starting points for this experiment. Neither establishes image-recognition accuracy, production agent experience, or live-provider quality.

Public sources

- First-party sources, checked September 17, 2026:
- [Software Engineer, Agents](#)
 - [Introducing Data Analyst Loops](#)
 - [Building Data Analyst Loops](#) (indexed text; direct cache fetch failed)
 - [API Reference](#)